BÖLÜM 10 / MODULE 10

MAANG Eşzamanlılık Mülakatı & Sistem Tasarımı Ustalığı

MAANG Concurrency Whiteboard & System Design Mastery

MAANG mülakatlarında sorulan 4 efsanevi sistem tasarımı vakası: Thread-Safe O(1) LRU Cache, LMAX Disruptor RingBuffer kilit-siz mesajlaşma, Token-Bucket & Leaky-Bucket hız sınırlayıcı, HikariCP FastPath havuz mekaniği; 45 dakikalık mülakat stratejisi ve Staff+ mimar değerlendirme kriterleri.

Four legendary MAANG concurrency whiteboard challenges: Thread-Safe O(1) LRU Cache, LMAX Disruptor lock-free RingBuffer, Token-Bucket & Leaky-Bucket rate limiting, HikariCP ConcurrentBag connection pool architecture; plus the 45-minute interview blueprint and Staff+ architect evaluation criteria.

55

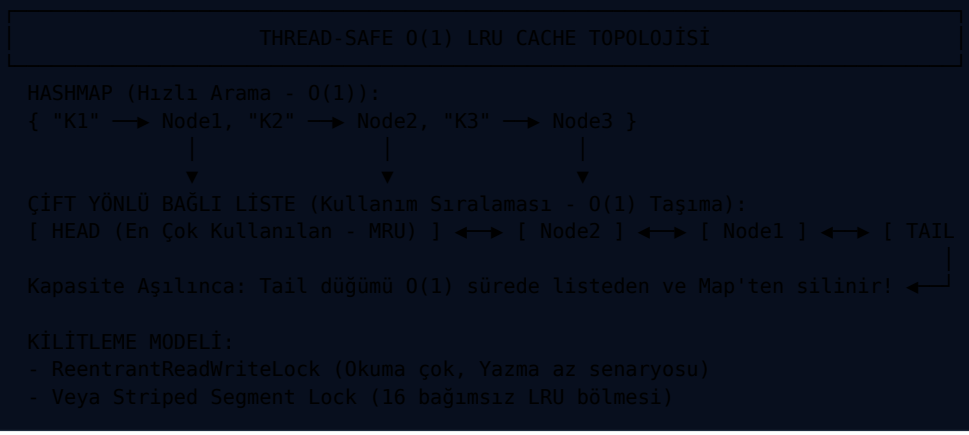
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 1: Thread-Safe O(1) LRU Cache Mimarisi (ReentrantReadWriteLock + Çift Bağlı Liste)

MAANG Case 1: Thread-Safe O(1) LRU Cache (Lock Striping & Linked Nodes)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphane masası düşünün. Masada en fazla 5 kitap durabiliyor. Birisi masadaki bir kitabı okuduğunda, o kitabı alıp masanın en sağına ('en son okunanlar' tarafına) koyarsınız. Eğer masaya yeni 6. bir kitap gelirse, masanın en solunda günlerdir kimsenin kapağını açmadığı en eski tozlu kitabı (`LRU - Least Recently Used`) çöpe atarsınız. Kitabı saniyede bulmak için bir fihrist (HashMap), sıralamayı anında güncellemek için ise iki yönlü bir tren vagonu zinciri (Doubly Linked List) kullanırız!

🇬🇧 ENGLISH (ELI5)

Imagine a coffee table that holds exactly 5 books. Whenever someone reads a book, you immediately slide it to the far right ('Most Recently Used'). If someone brings a 6th book and the table is full, you discard the book sitting at the far left that hasn't been touched in weeks ('Least Recently Used'). To find books in $O(1)$ time, we use an index card catalog (`HashMap`). To move books around in $O(1)$ time, we use a doubly linked train of cards (`DoublyLinkedList`). Both read and write operations maintain sub-millisecond thread safety!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesinde LRU Cache Tasarımı: 1) Veri Yapısı Seçimi: Yalnızca `HashMap` kullanılsaydı en eski elemanı bulmak $O(N)$ sürerdi. Yalnızca `LinkedList` kullanılsaydı arama $O(N)$ olurdu. `HashMap\<K, Node>` ile `DoublyLinkedList` birleştirildiğinde hem arama hem güncelleme $O(1)$ olur. 2) Eşzamanlılık Zafiyeti: LRU Cache'te `get(K)` işlemi MASUM BİR OKUMA DEĞİLDİR! Okunan elemanın listenin başına (`MRU`) taşınması gerektiği için liste üzerinde yapısal bir YAZMA (mutation) gerçekleşir. Bu yüzden saf `ReadLock` kullanılamaz; taşıma anında `WriteLock` gerekir. 3) Ölçeklenebilirlik Çözümü: Caffeine Cache gibi endüstri lideri kütüphaneler bu sorunu RingBuffer ve aktarım kuyrukları (Lossy / Event-Driven buffers) ile kilit almadan çözer. Mülakatta `ReentrantReadWriteLock` ile başlayıp Lock-Free buffer optimizasyonunu açıklamak Staff+ puanı kazandırır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Architectural Breakdown: 1) Structural Duality: Combining `HashMap\<K, Node\<K,V>>` with an internal Doubly-Linked List guarantees $O(1)$ key lookup and $O(1)$ eviction/re-linking. 2) The Read Mutation Trap: In LRU semantics, `get(key)` is NOT a read-only operation! Accessing an entry requires relocating its node to the Head (MRU), inducing structural mutations to adjacent pointer references. Consequently, standard read-locks fail; writers must acquire exclusive control. 3) Production Optimization: Explain to the interviewer that state-of-the-art caches (like Ben Manes' Caffeine) decouple reads from evictions using lock-free thread-local read buffers and a bounded MPSC ring-buffer to batch eviction updates asynchronously.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Thread-Safe LRU Cache Referans Uygulaması
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ThreadSafeLruCache<K, V> {
    private static class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null); // Dummy head
    private final Node<K, V> tail = new Node<>(null, null); // Dummy tail
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();

    public ThreadSafeLruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        rwLock.writeLock().lock(); // Erişim sıralaması değiştiği için write lock şart!
        try {
            Node<K, V> node = map.get(key);
            if (node == null) return null;
            moveToHead(node);
            return node.value;
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    public void put(K key, V value) {
        rwLock.writeLock().lock();
        try {
            Node<K, V> node = map.get(key);
            if (node != null) {
                node.value = value;
                moveToHead(node);
            } else {
                if (map.size() >= capacity) {
                    Node<K, V> lru = removeTail();
                    map.remove(lru.key);
                }
                Node<K, V> newNode = new Node<>(key, value);
                map.put(key, newNode);
                addToHead(newNode);
            }
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
    private void moveToHead(Node<K, V> node) {
        removeNode(node);
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatta LinkedHashMap(capacity, loadFactor, true) miras alarak removeEldestEntry metodunu ezmeyi önerebilirsiniz; ancak bu sınıf synchronized DEĞİLDİR! Collections.synchronizedMap() ile sarsanız bile kilit haritanın tamamını kilitler ve ölçeklenemez.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Subclassing `LinkedHashMap` and overriding `removeEldestEntry` is valid for rapid prototyping, but `LinkedHashMap` is not thread-safe. Wrapping it with `Collections.synchronizedMap()` imposes a coarse-grained bottleneck across the entire map, failing MAANG scalability reviews.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• LRU Cache'te okuma işlemi elemanın yerini değiştirdiği için yapısal bir yazmadır. HashMap + DoublyLinkedList birleşimi O(1) sağlar.

🎯 Key Takeaways for the Interview

• In LRU caches, reads mutate node order. Coupling a HashMap with a Doubly-Linked List guarantees \$O(1)\$ lookups and relocations under lock synchronization.

56

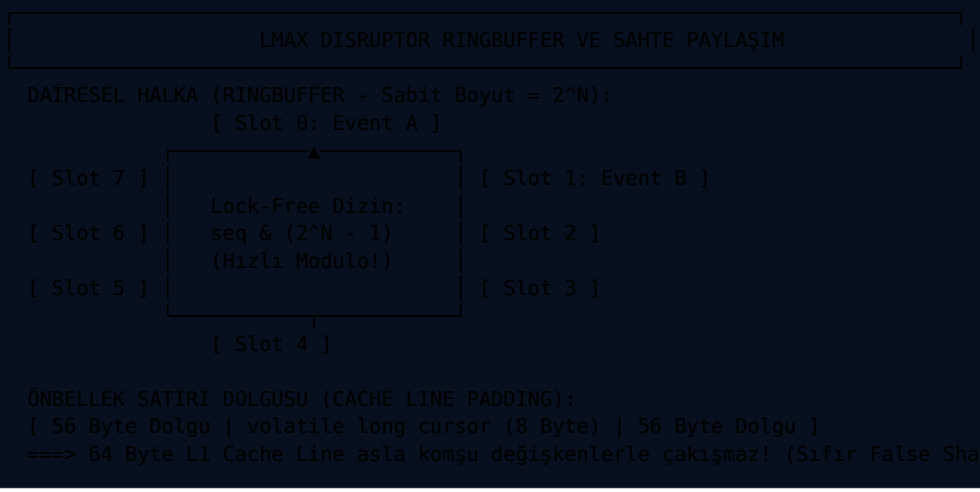
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 2: LMAX Disruptor Deseni & Lock-Free RingBuffer ile Ultra-Düşük Gecikme

MAANG Case 2: LMAX Disruptor & Lock-Free RingBuffer with Cache Padding

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Disruptor, Londra borsasındaki saniyede 6 milyon emri sıfır gecikmeyle işlemek için icat edilmiştir. Klasik kuyruklar iki kişinin birbirinin eline mektup tutuşturması gibidir (biri verirken diğeri bekler). RingBuffer ise döner bir dönme dolaptır. Sepetler önceden hazırdır ve hiç durmadan döner. Üretici sıradaki boş sepete kâğıdı fırlatır, tüketici arkasından gelip alır; ne kilit vardır ne bekleme! Bellek önbelleklerini (CPU Cache) korumak için de değişkenlerin sağına soluna boş dolgu konur, böylece çekirdekler birbirini hiç rahatsız etmez.

🇬🇧 ENGLISH (ELI5)

The LMAX Disruptor was engineered to process 6,000,000 orders/sec with nanosecond latency in financial exchanges. Standard queues use locks and dynamically allocate nodes, hammering the garbage collector. The Disruptor pre-allocates an immutable circular carousel (`RingBuffer`). Producers claim slots using monotonic atomic sequences, overwrite pre-allocated event slots in-place, and consumers read behind them with zero locks. Cache-line padding ensures CPU L1/L2 caches never suffer false sharing.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LMAX Disruptor'ın 4 Temel Mimari Sırrı: 1) Sıfır Çöp Üretimi (Zero-Allocation): RingBuffer boyutu 2'nin kuvvetidir (2^N). Tüm olay nesneleri uygulama başlarken heap'te önceden tahsis edilir (`pre-allocation`). Çalışma anında asla `new` çağrılmaz; aynı bellek yuvaları ezilerek yeniden kullanılır, GC duraklamaları (Stop-the-World) tamamen yok edilir. 2) İkili Maskeleme (Bitwise Indexing): Dizi indeksi hesaplanırken yavaş olan modulo (`%`) yerine `sequence & (bufferSize - 1)` bit düzeyinde VE işlemi kullanılır; tek bir CPU çevriminde slot bulunur. 3) Kilit-Siz Sıralama (Single-Writer Principle): Üretici sırayı tekil `volatile long cursor` ile takip eder; okuyucular CAS döngüsü olmadan `VarHandle.getAcquire` veya `Volatile` ile takip eder. 4) Sahte Paylaşım Önleme: Sıra değişkenleri arasına 56 baytlık dolgu (`padding`) yerleştirilerek değişkenin tek başına 64 baytlık L1 CPU Cache Line'ı kaplaması sağlanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Mechanical Sympathy of the LMAX Disruptor: 1) Zero Garbage Allocation: Array capacity is constrained to powers of two (2^N). Event slots are pre-allocated during initialization. Mutating state involves writing in-place to pre-existing slots, eliminating memory churn and GC pauses. 2) Bitwise Power-of-Two Indexing: Modulo arithmetic is replaced by bitwise masking (`sequence & (bufferSize - 1)`), executing in a single CPU clock cycle. 3) Lock-Free Mechanical Sympathy: Discards blocking queues and monitor locks in favor of monotonic sequence barriers backed by memory-fenced primitives. 4) Cache-Line Isolation: Sequence counters are isolated on separate 64-byte L1 cache lines using padding fields or `\@jdk.internal.vm.annotation.Contended`, preventing MESI cache invalidation storms.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Basitleştirilmiş Lock-Free RingBuffer Prensibi
import java.util.concurrent.atomic.AtomicLong;

public class MinimalDisruptorRingBuffer<T> {
    private final Object[] ringBuffer;
    private final int mask;
    // Cache Line Padding simülasyonu
    private final AtomicLong cursor = new AtomicLong(-1);

    public MinimalDisruptorRingBuffer(int powerOfTwoSize) {
        if (Integer.bitCount(powerOfTwoSize) != 1) {
            throw new IllegalArgumentException("Boyut 2'nin kuvveti olmalıdır!");
        }
        this.ringBuffer = new Object[powerOfTwoSize];
        this.mask = powerOfTwoSize - 1;
    }

    public long nextWriteSequence() {
        return cursor.incrementAndGet();
    }

    public void publish(long seq, T event) {
        int index = (int) (seq & mask); // Bitwise modulo (Işık hızında!)
        ringBuffer[index] = event;
    }

    @SuppressWarnings("unchecked")
    public T read(long seq) {
        int index = (int) (seq & mask);
        return (T) ringBuffer[index];
    }

    void main() {
        var buffer = new MinimalDisruptorRingBuffer<String>(1024);
        long seq = buffer.nextWriteSequence();
        buffer.publish(seq, "BIST100_LIMIT_ORDER_BUY_1000");

        String order = buffer.read(seq);
        System.out.println("RingBuffer'dan Okunan Emir: " + order);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

RingBuffer boyutu kesinlikle 2'nin kuvveti (1024, 4096, 65536) olmalıdır! Aksi halde `seq & (size - 1)` formülü çalışmaz ve dizi sınırları dışına taşma (ArrayIndexOutOfBoundsException) oluşur.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

RingBuffer capacity MUST strictly be a power of two (2^N). If breached, the bitwise indexing equation `sequence & (capacity - 1)` produces illegal indices, causing runtime exceptions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

🎯 Key Takeaways for the Interview

• Disruptor sıfır çöp tahsisi, 2'nin kuvveti bit maskelemesi ve cache-line padding ile kilit almadan mikrosaniye altı gecikme üretir.

• The Disruptor achieves sub-microsecond latency through zero GC allocation, bitwise power-of-two ring indexing, and false-sharing cache padding.

57

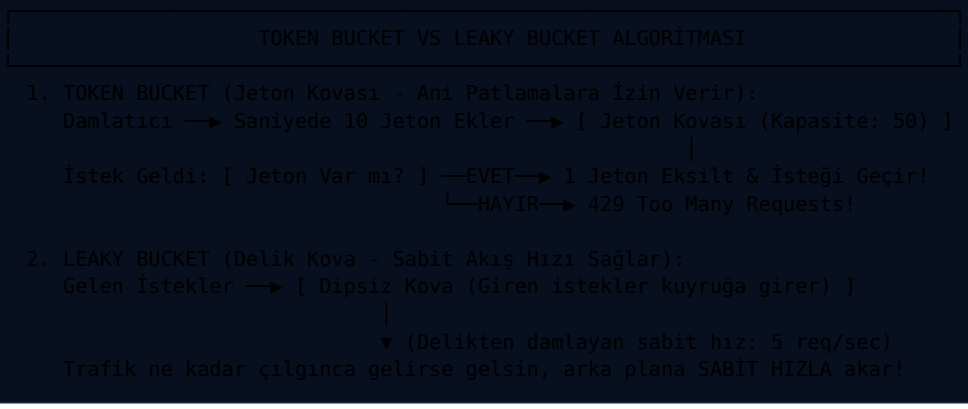
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 3: Dağıtık Token-Bucket & Leaky-Bucket Hız Sınırlayıcı (Rate Limiter)

MAANG Case 3: Distributed Token-Bucket & Leaky-Bucket Rate Limiter

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Token Bucket bir lunapark biletçisi gibidir. Kovaya her saniye 1 bilet atılır (en fazla 10 bilet birikir). Bir anda 5 arkadaş gelirseniz kovadaki 5 bileti alıp hemen dönme dolaba binebilirsiniz (patlamalı trafiğe izin verir). Bilet biterse kapıda kalırsınız. Leaky Bucket ise altı delik bir su kovasıdır. Kovaya yukarıdan bir kova suyu birden döksen de altındaki minik delikten su damla damla, sabit ve sakin bir hızla akar; arka plandaki narin veritabanını asla boğmaz!

🇬🇧 ENGLISH (ELI5)

Token Bucket is an arcade coin dispenser. It drops 1 token every second into a cup that holds up to 10 tokens. A burst of 5 gamers can grab 5 tokens simultaneously and play immediately (burst-tolerant). Once empty, newcomers are rejected. Leaky Bucket is a funnel with a tiny hole at the base. Even if a sudden flood of 100 gallons dumps into the funnel, water drips out of the bottom at a strictly constant rate of 1 drop/second, shielding fragile downstream services from traffic spikes.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Mülakatında Algoritma Karşılaştırması ve Kilit-Siz Uygulama: 1) Token Bucket Mantığı: İki ana değişken vardır: `capacity` ve `refillRate`. Her saniye bir zamanlayıcı thread çalıştırmak yerine 'Tembel Doldurma' (Lazy Refill) formülü uygulanır: `yeniJeton = min(kapasite, mevcutJeton + (simdikiZaman - sonYenilemeZamani) × refillRate)`. Bu sayede kova kontrolü yapay zeka gibi sıfır arka plan maliyetiyle yürütülür. 2) CAS ile Eşzamanlılık: `AtomicLong` veya `AtomicReference` kullanılarak jeton harcama işlemi kilit almadan tek çevrimde gerçekleştirilir. 3) Leaky Bucket Mantığı: Genellikle `ArrayBlockingQueue` veya Redis kuyruğu ile modellenir; gelen istekler kuyruğa alınır, bir tüketici thread sabit hızda (`rate`) kuyruktan iş çeker.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Algorithmic Breakdown: 1) Token Bucket Optimization: Avoid dedicated timer background threads. Implement 'Lazy Refill': compute accumulated tokens mathematically on-demand: `newTokens = min(capacity, tokens + (now - lastRefill) × rate)`. 2) Lock-Free State Mutation: Encapsulate token counts and timestamps in an immutable Java Record updated atomically via `AtomicReference.compareAndSet()`. 3) Leaky Bucket Topology: Models traffic shaping. Incoming requests enter a bounded FIFO buffer; an internal rate-governor drains requests at an uncompromising cadence, smoothing bursty traffic profiles before downstream ingress.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Lock-Free ve Tembel Doldurmalı Token Bucket Rate Limiter
import java.time.Instant;
import java.util.concurrent.atomic.AtomicReference;

public class AtomicTokenBucketLimiter {
    private final long capacity;
    private final double refillTokensPerSecond;

    // Durumu değişmez bir record içinde atomik tut
    private record State(long tokens, long lastRefillNanos) {}
    private final AtomicReference<State> state;

    public AtomicTokenBucketLimiter(long capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.state = new AtomicReference<>(new State(capacity, System.nanoTime()));
    }

    public boolean tryAcquire(long tokensRequested) {
        while (true) {
            State current = state.get();
            long now = System.nanoTime();
            long elapsedNanos = Math.max(0, now - current.lastRefillNanos());
            double newTokens = (elapsedNanos / 1_000_000_000.0) * refillTokensPerSecond;

            long updatedTokens = Math.min(capacity, current.tokens() + (long) newTokens);

            if (updatedTokens < tokensRequested) {
                return false; // Jeton yetersiz: İstek reddedildi (429)!
            }

            State next = new State(updatedTokens - tokensRequested, now);
            if (state.compareAndSet(current, next)) {
                return true; // Atomik olarak jeton düşüldü: İstek geçti!
            }
        }
    }

    void main() {
        var limiter = new AtomicTokenBucketLimiter(5, 2.0); // 5 kapasite, sn'de 2 jeton
        for (int i = 1; i <= 7; i++) {
            boolean allowed = limiter.tryAcquire(1);
            System.out.printf("İstek #%d Kabul Edildi mi? %s%n", i, allowed ? "✅ EVET" : "❌ 429
RED");
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Dağıtık sistemlerde (ör. Kubernetes üzerinde 50 pod) lokal RAM'deki Token Bucket işe yaramaz. Durumu Redis üzerinde Lua Script ile atomik yürütmeli veya NGINX / Envoy API Gateway katmanında merkezileştirmelisiniz.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Local in-memory rate limiters fail across horizontal deployments (e.g. 50 microservice pods). Synchronize global rate limiting states using atomic Redis Lua scripts or offload enforcement to edge gateways like Envoy or Cloudflare.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Token Bucket ani trafik patlamalarına izin verir (lazy refill ile kilit-siz hesaplanır). Leaky Bucket ise trafiği sabit hızda damlatarak pürüzsüzleştirir.

🎯 Key Takeaways for the Interview

• Token Bucket tolerates bursty ingress computed via lock-free lazy refills. Leaky Bucket smooths spikes into a constant-rate outflow.

58

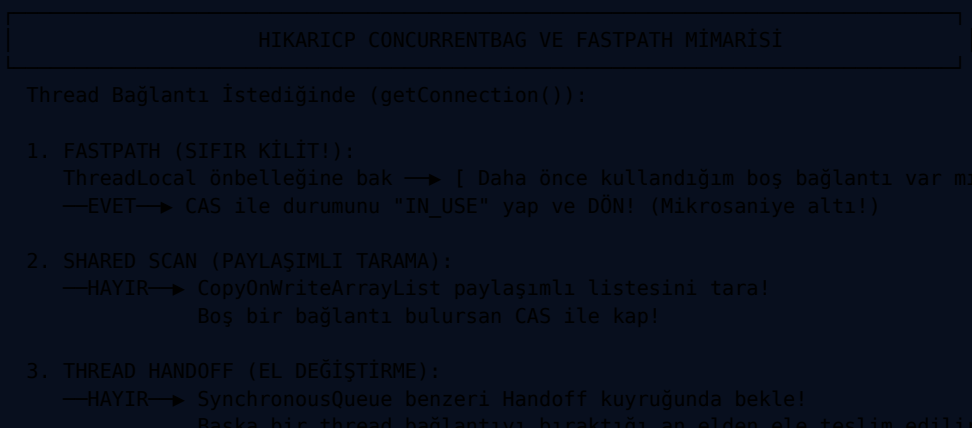
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 4: HikariCP FastPath Bağlantı Havuzu Mekaniği (ConcurrentBag & ThreadLocal Handoff)

MAANG Case 4: HikariCP FastPath Connection Pool (ConcurrentBag Architecture)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

HikariCP dünyanın en hızlı veritabanı bağlantı havuzudur. Eski havuzlar tek bir kapıcı koyar; 100 kişi o kapıcının önünde sıraya girerdi (`synchronized`). HikariCP ise şöyle der: 'Dün kullandığın anahtarı kendi cebine (`ThreadLocal`) koy. Yarın tekrar gelirsen kapıcıya hiç sormadan cebindeki anahtarı çıkarıp doğrudan içeri gir!' Cebinde anahtar yoksa masaya bakar, masada da yoksa anahtarı getirecek ilk kişiyi el sıkışma noktasında bekler!

🇬🇧 ENGLISH (ELI5)

HikariCP is universally recognized as the fastest JDBC connection pool on earth. Legacy pools (e.g. DBCP, c3p0) guarded connections behind a single synchronized gate. HikariCP implemented `ConcurrentBag`: each thread caches its most recently used connection in a `ThreadLocal`. When requesting a connection, it inspects its private thread cache first—if idle, it claims it with zero lock contention! If absent, it scans the shared bag; if empty, it registers on a zero-capacity handoff queue.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HikariCP ConcurrentBag'in 3 Katmanlı Çalma Hiyerarşisi: 1) Katman 1 (ThreadLocal FastPath): `threadLocalBagEntry.get()` çağrılır. Eğer referans varsa ve durumu `STATE_NOT_IN_USE` ise, tek bir atomik `CAS(STATE_NOT_IN_USE -> STATE_IN_USE)` ile kilit almadan bağlantı tahsis edilir. CPU context switch ve kilit çekişmesi sıfırdır. 2) Katman 2 (Shared List Scanning): ThreadLocal boşsa veya başka bir thread tarafından çalınmışsa, havuz `CopyOnWriteArrayList` üzerinde dolaşarak uygun boş bağlantıları CAS ile arar. 3) Katman 3 (Synchronous Handoff): Havuz tamamen meşgulse çağıran thread `SynchronousQueue` benzeri adil bir teslim kuyruğunda bekletilir. Başka bir thread `close()` çağırdığında bağlantıyı havuza iade etmek yerine doğrudan bekleyen thread'e teslim eder (Thread-to-Thread Handoff).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HikariCP ConcurrentBag Triple-Tiered Resolution: 1) Tier 1 (ThreadLocal FastPath): Queries thread-local cache. If a previously utilized entry is present in state `STATE_NOT_IN_USE`, an atomic CAS transitions state to `STATE_IN_USE`. Total time: nanoseconds, zero lock contention. 2) Tier 2 (Shared Bag Sweep): If the ThreadLocal is vacant or stolen, the thread iterates over a lock-free `CopyOnWriteArrayList` of pool entries, attempting CAS acquisitions. 3) Tier 3 (Synchronous Handoff Queuing): If all connections are occupied, the thread parks on a specialized `SynchronousQueue` handoff. When a loaner thread invokes `close()`, instead of recycling the entry to the shared list, it hands the connection off directly to the waiting worker thread.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: HikariCP ConcurrentBag FastPath Mantığının Minyatür Modeli
import java.util.concurrent.atomic.AtomicInteger;

public class MiniConcurrentBagPool {
    private static final int STATE_NOT_IN_USE = 0;
    private static final int STATE_IN_USE = 1;

    public static class BagEntry {
        final String connectionId;
        final AtomicInteger state = new AtomicInteger(STATE_NOT_IN_USE);
        public BagEntry(String id) { this.connectionId = id; }
    }

    // Her thread'in son kullandığı bağlantıyı tutan FastPath önbelleği
    private final ThreadLocal<BagEntry> fastPathCache = new ThreadLocal<>();
    private final BagEntry[] sharedBag;

    public MiniConcurrentBagPool(int size) {
        sharedBag = new BagEntry[size];
        for (int i = 0; i < size; i++) {
            sharedBag[i] = new BagEntry("CONN_JDBC_" + i);
        }
    }

    public BagEntry borrowConnection() {
        // 1. ADIM: FastPath (ThreadLocal'dan kilit-siz al!)
        BagEntry cached = fastPathCache.get();
        if (cached != null && cached.state.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            System.out.println("⚡ FastPath ile anında alındı: " + cached.connectionId);
            return cached;
        }

        // 2. ADIM: Paylaşımlı havuzu tara
        for (BagEntry entry : sharedBag) {
            if (entry.state.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
                fastPathCache.set(entry); // Gelecek sefer için önbelleğe al!
                System.out.println("🔍 Shared Bag'den bulundu: " + entry.connectionId);
                return entry;
            }
        }
        throw new RuntimeException("Tüm bağlantılar dolu (Havuz Tükendi)!");
    }

    public void releaseConnection(BagEntry entry) {
        entry.state.set(STATE_NOT_IN_USE);
    }

    void main() {
        var pool = new MiniConcurrentBagPool(5);
        var c1 = pool.borrowConnection();
        pool.releaseConnection(c1);
        // İkinci istek FastPath üzerinden yıldırım hızında gelir!
        var c2 = pool.borrowConnection();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

HikariCP'de havuz boyutunu 1000 yapmak bağlantı hızını ARTIRMAZ, aksine veritabanı disk I/O ve CPU'sunu context switch ile boğar! HikariCP formülü: `pool_size = Tn × (CoreCount × 2 + EffectiveSpindleCount)` şeklindedir. Çoğu MAANG servisi için 10-20 bağlantı binlerce eşzamanlı isteği karşılamaya yeterlidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Configuring pool sizes to 500+ does NOT improve throughput; it cripples database CPU and disk subsystems with context-switching thrash. Follow the HikariCP sizing formula: `pool_size = CoreCount × 2 + SpindleCount`. For typical services, a modest pool of 10-20 connections out-scales bloated pools by 10x.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• HikariCP'nin sırrı ConcurrentBag'dir: ThreadLocal FastPath ile önce kendi önbelleğine bakar, yoksa paylaşımlı havuzdan CAS ile çalar.

🎯 Key Takeaways for the Interview

• HikariCP's supremacy lies in ConcurrentBag: ThreadLocal FastPath bypasses locks; shared sweeps and direct handoffs eliminate pool queuing.

59

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

Eşzamanlılık Beyaz Tahta Planı: 45 Dakikalık MAANG Mülakat Yönetim Stratejisi

Concurrency Whiteboard Blueprint: 45-Minute MAANG Execution Strategy

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                45 DAKİKALIK MAANG CONCURRENCY MÜLAKAT ZAMAN ÇİZELGESİ

00-05 dk: GEREKSİNİMLERİ NETLEŞTİR (Clarify Requirements & Thread Safety Scope)
05-12 dk: VERİ YAPILARI & ÇEKİŞME ANALİZİ (Data Structures & Contention Points)
12-25 dk: ÇALIŞAN İLK KODU YAZ (Baseline Thread-Safe Code - Monolithic Lock)
25-35 dk: KİLİTSİZ / DETAYLI KİLİT OPTİMİZASYONU (Fine-Grained / CAS / Disruptor)
35-42 dk: SINIR DURUMLAR & TEŞHİS (Deadlock, Spurious Wakeups, Memory Leaks)
42-45 dk: ÖZET & STAFF+ MİMAR PERSPEKTİFİ (Metrics, Loom Virtual Threads, Tuning)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

MAANG mülakatı bir boks maçına benzer. Ringe çıkar çıkmaz gözün kapalı yumruk sallarsan (soruyu dinlemeden hemen kod yazmaya başlarsan) ilk rauntta nakavt olursun! Önce rakibini süz (soruyu sor: 'Kaç thread yazacak? Okuma mı çok yazma mı?'). Sonra gardını alıp sağlam bir vuruş yap (önce basit ama çalışan kilitli kodu yaz). Sonra özel hareketini göster (Lock-Free veya Virtual Thread optimizasyonunu açıkla). Hakemler şovuna hayran kalır!

🇬🇧 ENGLISH (ELI5)

A MAANG concurrency interview is like a surgical operation. If you grab the scalpel and start cutting before reviewing the patient's vitals (writing code without clarifying read/write contention ratios), the patient dies. First clarify invariants and boundaries. Second, provide a working baseline using clean abstractions. Third, optimize hot paths using fine-grained locks or lock-free CAS. Finally, walk through edge cases like thread starvation, spurious wakeups, and observability.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Mülakatçının Not Defteri ve 5 Kritik Değerlendirme Ekseni: 1) Soru Sorma (Requirement Clarification): 'Sistemimiz read-heavy mi yoksa write-heavy mi? Gecikme toleransı (SLA) nedir? Thread sayısı sabit mi yoksa dinamik mi?' sorularını sormak adayı anında Senior havuzuna taşır. 2) İlerlemeli Tasarım (Progressive Enhancement): Doğrudan aşırı karmaşık lock-free kod yazmaya çalışmayın! Önce `synchronized` veya `ReentrantLock` ile %100 doğru çalışan bir temel (baseline) kurun. Ardından 'Burada çekişme olursa Striped Locking veya ConcurrentHashMap'e evrilebiliriz' diyerek optimizasyona geçin. 3) Bellek Modeli Hissiyatı (JMM Fluency): Kod yazarken `volatile`, happens-before ve görünürlük etkilerini sesli olarak mülakatçıya anlatın. 4) Köşe Vakalar (Edge Cases): Spurious wakeups, interrupt durumunda thread bayrağını koruma (`Thread.currentThread().interrupt()`), ve kilitlerin `finally` içinde bırakılması adayın reflekslerini gösterir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Interviewer Scoring Rubric: 1) Clarification Phase: Proactively inquire about read-to-write ratios, thread counts, latency budgets, and memory constraints. 2) Progressive Implementation: Never dive prematurely into unverified lock-free wizardry. Implement an unassailable baseline using idiomatic locks first, then systematically profile and optimize hotspots to demonstrate maturity. 3) Mechanical & JMM Fluency: Articulate happens-before edges, CPU memory fences, and cache-line effects aloud while drafting variables. 4) Defensive Rigor: Always restore interrupted statuses, guarantee lock releases in `finally` blocks, and guard condition checks within `while` loops against spurious wakeups.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: MAANG Mülakatında Sunulacak İdeal Şablon Kodu
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class MaangInterviewTemplate<T> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();
    private boolean ready = false;

    public void waitForSignal() throws InterruptedException {
        lock.lock();
        try {
            // ⚠ MÜLAKAT KURALI 1: if değil daima WHILE kullan (Spurious Wakeup Savunması!)
            while (!ready) {
                condition.await();
            }
            System.out.println("Sinyal alındı, kritik operasyon devam ediyor...");
        } finally {
            // ⚠ MÜLAKAT KURALI 2: Kilit DAİMA finally içinde açılır!
            lock.unlock();
        }
    }

    public void sendSignal() {
        lock.lock();
        try {
            ready = true;
            condition.signalAll(); // Tüm bekleyenleri haberdar et
        } finally {
            lock.unlock();
        }
    }

    void main() {
        System.out.println("MAANG şablonu hazır.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatçının 'Neden synchronized yerine ReentrantLock kullandın?' sorusuna hazırlıklı olun. Yanıtınız: tryLock (timeout), adil kilit seçeneği, çoklu Condition kuyruğu ve kesilebilir (lockInterruptibly) kilitleme yetenekleri olmalıdır.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Be prepared for the inevitable question: 'Why prefer ReentrantLock over synchronized?' Your crisp answer: timed polling via `tryLock()`, interruptible lock acquisition, fairness flags, and multiple independent `Condition` wait-queues.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

🎯 Key Takeaways for the Interview

• Mülakata önce gereksinimleri netleştirerek başlayın. Basit ve doğru bir kilitli temel kurun, ardından lock-free optimizasyonları ve köşe vakaları tartışın.

• Begin by clarifying contention ratios. Implement a robust baseline first, then progressively optimize with lock-free primitives and edge-case defenses.

60

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

Staff+ Concurrency Mimarı: Canlı Sistem Çöküş Otopsisi & Çok Çekirdekli Ayarlar

The Staff+ Concurrency Architect: Production Post-Mortems & Tuning

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



```
                STAFF+ MİMARININ CANLI SİSTEM MÜDAHALE HARİTASI

CANLI SİSTEM ÇÖKÜŞÜ (INCIDENT):
- Semptom: 99.9th Percentile Gecikme 50ms'den 12 Saniyeye Fırladı!
- CPU Kullanımı: %98 (128 Çekirdekli Sunucu)

ADLİ PROTOKOL (POST-MORTEM WORKFLOW):
1. METRİKLERİ İNCELE  ──► JVM GC Pauses? (ZGC / Shenandoah kontrolü)
2. THREAD DUMP AL     ──► jcmd <PID> Thread.print ──► Kilit çekişmesi var mı?
3. PROFİLER BAĞLA     ──► async-profiler ──► Cache Misses & Lock Contention
4. MİMARİ MÜDAHALE:
   - Kaba kilitleri Striped Lock veya Lock-Free'e çevir
   - Platform thread havuzlarını Project Loom Sanal Thread'lerine göç ettir
   - False Sharing olan kritik alanlara @Contended dolgusu uygula!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir çırak, araba bozulunca 'Motoru değiştirelim' der. Kıdemli bir usta ise kulağını motora dayar, 3. silindirin contasındaki 1 milimlik yırtığı saniyesinde bulur. Staff+ Concurrency Mimarı, binlerce sunucunun saniyede milyonlarca işlem yaptığı bir MAANG şirketinde sistem kilitlendiğinde paniklemez. 3 dakikada thread dump alır, hangi thread'in hangi kilit yüzünden tıkandığını teşhis eder ve sistemi sıfır veri kaybıyla ayağa kaldırır!

🇬🇧 ENGLISH (ELI5)

A novice engineer sees an outage and suggests restarting all servers. A Staff+ Concurrency Architect approaches a production failure like a master forensic detective. With a 128-core server melting down at 99% CPU and p99 latency spiking to 10 seconds, the Staff Architect captures continuous low-overhead async-profiler flame graphs, identifies a single contested cache line or lock convoy in under 3 minutes, and executes a targeted hot-patch.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Staff+ Seviyesinde Üretim Optimizasyonu ve Donanım Uyumu (Mechanical Sympathy): 1) async-profiler ile Saftepoint Bias Önleme: Standart Java profiler'ları thread'leri yalnızca Safepoint anlarında örnekler; bu da yanıltıcı sonuçlar verir. `async-profiler` Linux çekirdeğinin `perf_events` alt yapısını kullanarak safepoint bağımsız gerçek CPU döngülerini ve kilit bekleme alev grafiklerini (Flame Graph) çizer. 2) NUMA (Non-Uniform Memory Access) Duyarlılığı: Çok soketli sunucularda bir CPU çekirdeğinin diğer soketin RAM'ine erişmesi yüksek gecikmeye yol açar. JVM `-XX:+UseNUMA` bayrağı ile donatılmalıdır. 3) Modern Çöp Toplayıcılar (ZGC & Shenandoah): Yüksek eşzamanlı sistemlerde STW (Stop-The-World) duraklamalarını 1 milisaniyenin altına indirmek için Generational ZGC (`-XX:+UseZGC`) tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Staff+ Architectural Mastery & Mechanical Sympathy: 1) Safepoint-Bias-Free Profiling: Standard JVM profiler samples exclusively at safepoint intervals, introducing severe observational skew. Deploy `async-profiler` leveraging Linux kernel `perf_events` to construct accurate off-CPU and lock contention flamegraphs. 2) NUMA Node Awareness: On multi-socket NUMA hardware architectures, cross-socket memory bus hops inflate latency. Enable `-XX:+UseNUMA` to co-locate thread allocations within local memory banks. 3) Sub-Millisecond GC Engine: Deploy Generational ZGC (`-XX:+UseZGC`) to eliminate Stop-The-World latency jitter, capping max GC pauses at under 1 millisecond across multi-terabyte heaps under heavy concurrent throughput.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Staff+ Seviyesi - Metrik Toplama ve Sağlık Denetimi
import java.lang.management.ManagementFactory;
import java.lang.management.OperatingSystemMXBean;
import java.lang.management.ThreadMXBean;

public class StaffArchitectDiagnostics {
    public static void printSystemHealthReport() {
        OperatingSystemMXBean os = ManagementFactory.getOperatingSystemMXBean();
        ThreadMXBean threads = ManagementFactory.getThreadMXBean();

        System.out.println("================================================");
        System.out.println("🛡️ STAFF+ PRODUCTION HEALTH MONITORING REPORT");
        System.out.println("================================================");
        System.out.printf("Mevcut CPU Yükü: %.2f%%%n", os.getSystemLoadAverage() * 100);
        System.out.printf("Aktif Canlı Thread Sayısı: %d%n", threads.getThreadCount());
        System.out.printf("Zirve Thread Sayısı: %d%n", threads.getPeakThreadCount());
        System.out.printf("Toplam Başlatılan Thread: %d%n", threads.getTotalStartedThreadCount());

        long[] deadlocks = threads.findDeadlockedThreads();
        System.out.println("Deadlock Durumu: " + (deadlocks == null ? "SIFIR (Temiz)" : "🧯 " +
deadlocks.length + " Thread Kilitli!"));
        System.out.println("================================================");
    }

    void main() {
        printSystemHealthReport();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Canlı sistemde sorun çıkınca servisi hemen yeniden başlatmak (restart) kök nedeni sonsuza kadar yok eder! Restart atmadan önce MUTLAKA en az 3 adet thread dump ve heap dump (`jcmd \<PID> GC.heap_dump`) alın; delilleri koruyun.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Blindly restarting servers during a production outage destroys forensic evidence forever! Always automate taking 3 spaced thread dumps and a memory snapshot via `jcmd \<PID> GC.heap_dump` prior to restarting instances to preserve root-cause traces.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Staff+ mimar ezbere çözüm üretmez; metrikleri, thread dökümlerini ve donanım mimarisini (NUMA, Cache Line, ZGC) birleştirerek kök nedeni çözer.

🎯 Key Takeaways for the Interview

• A Staff+ architect marries software abstractions with hardware topology: profilers, lock contention traces, NUMA awareness, and sub-millisecond ZGC.